Databricks Interview FAQ Guide

Low, Medium and High Level Questions and Answers

105 questions covering Databricks, Delta Lake, Apache Spark, Unity Catalog, streaming, pipelines, performance, security, cost and real interview scenarios

Level	Who it is for	Questions
Level 1: LOW	Freshers, juniors, screening rounds	36
Level 2: MEDIUM	2 to 5 years, hands-on engineers	33
Level 3: HIGH	Senior, lead and architect roles	36

Note: Databricks product names change often (for example Workflows became Lakeflow Jobs, and Delta Live Tables became Lakeflow Declarative Pipelines). Where this matters, both names are given. Always confirm exact defaults and limits in the current Databricks documentation before an interview.

How to Use This Guide

Read the levels in order. Level 1 builds the vocabulary, Level 2 builds working skill, and Level 3 builds the depth and judgement that senior interviews test. Every answer is written the way you should say it aloud: a clear definition first, then how it works, then an example or code, then the trade-off.

A reliable 5-step formula for answering any question

- **Define** it in one sentence, in plain words.
- **Explain how it works** in two or three sentences, naming the key mechanism.
- **Give an example**: a command, a code snippet, or a real use case from your own work.
- **State the trade-off**: when you would not use it, the cost, or the limit.
- **Connect to experience**: "In my project we used this to ..." (always prepare real examples).

When you do not know the answer

Say what you do know, reason out loud from first principles, and state how you would verify it ("I would check the Spark UI for spill and skew, then look at the table file sizes"). Interviewers value structured thinking more than a perfect memory. Never invent facts; it is better to say "I have not used that feature in production, but I understand it works like this".

Contents

Level 1: LOW (Foundations) 4

Section A: Platform Basics 4

Section B: Compute 5

Section C: Notebooks and Developer Basics 7

Section D: Delta Lake Basics 9

Section E: Apache Spark Basics 12

Section F: Governance, Orchestration and Good Practices 13

Level 2: MEDIUM (Practitioner) 16

Section A: Delta Lake in Depth 16

Section B: Ingestion and Structured Streaming 19

Section C: Pipelines and Orchestration 21
Section D: Unity Catalog and Governance 22
Section E: Spark Performance and Internals 23
Section F: Operations, Cost and Delivery 25

Level 3: HIGH (Advanced and Architect) 27

Section A: Delta Lake and Engine Internals 27
Section B: Performance Scenarios 30
Section C: Advanced Streaming and Ingestion 32
Section D: Data Modelling, Pipelines and Governance Design 34
Section E: Security, Sharing and Migration 36
Section F: Operations, Cost, Delivery and Leadership 39
Section G: Scenario and Behavioural Questions 41

Appendix A: Rapid-Fire Reference (Defaults and Numbers) 44

Appendix B: Final Interview Checklist 45

Level 1: LOW (Foundations)

Fundamentals every candidate must know cold: platform basics, compute, notebooks, Delta basics, Spark basics and governance basics. Typical for freshers, juniors and screening rounds.

Section A: Platform Basics

Q1. What is Databricks, in simple words?

Databricks is a unified, cloud-based data and AI platform built around Apache Spark. It gives one place where data engineers, data analysts, data scientists and ML engineers can ingest data, transform it, query it with SQL, build dashboards, train machine-learning models and govern everything with a single security layer.

It was founded in 2013 by the original creators of Apache Spark (from UC Berkeley's AMPLab). The company also created or leads several open-source projects: **Apache Spark**, **Delta Lake**, **MLflow**, **Unity Catalog** and **Delta Sharing**.

Interview tip: say "it is a managed Lakehouse platform that runs on AWS, Azure and GCP, so the team does not have to build or operate Spark clusters by hand."

Q2. What is a Lakehouse? How is it different from a data warehouse and a data lake?

A **data warehouse** stores structured data in a proprietary format, is fast for SQL and BI, and gives ACID transactions, but it is expensive and weak for unstructured data and machine learning.

A **data lake** stores any kind of file (CSV, JSON, Parquet, images, logs) cheaply on object storage such as S3 or ADLS, but plain files have no transactions, no schema enforcement and poor governance, so lakes often turn into "data swamps".

A **Lakehouse** combines both: open-format files on cheap cloud storage, plus a table layer (Delta Lake) that adds ACID transactions, schema enforcement, time travel and good SQL performance, plus a governance layer (Unity Catalog). One copy of the data serves BI, data engineering, streaming and AI.

Q3. What is the difference between the control plane and the compute plane?

This is the basic Databricks architecture question.

- **Control plane**: managed by Databricks in its own cloud account. It hosts the web application, notebooks (code and results metadata), job scheduler, cluster manager, Unity Catalog metadata and REST APIs.
- **Compute plane**: where your data is actually processed. In **classic compute** the clusters run in *your* cloud account and your virtual network, so data stays in your environment. In **serverless compute** the compute runs in a Databricks-managed account, and Databricks handles provisioning and scaling.

Your data itself lives in your own cloud storage (S3, ADLS Gen2 or GCS), not inside Databricks.

Q4. What are the main components of a Databricks workspace?

A workspace is the web environment where you work. Its main parts are:

- **Workspace / Git folders**: notebooks, files and Git-connected repos.
- **Compute**: all-purpose clusters, job clusters, pools and SQL warehouses.
- **Catalog (Unity Catalog)**: catalogs, schemas, tables, volumes, functions and models.
- **Jobs and Pipelines**: scheduled and triggered orchestration (Lakeflow Jobs, formerly Workflows) and declarative pipelines.
- **SQL**: SQL editor, queries, dashboards and alerts.
- **Machine learning**: experiments, model registry, feature tables and model serving.

Q5. Which clouds does Databricks run on?

Databricks runs on **AWS**, **Microsoft Azure** and **Google Cloud**. On Azure it is a first-party Microsoft service (Azure Databricks), integrated with Microsoft Entra ID (formerly Azure AD), ADLS Gen2, Key Vault and Power BI. The core concepts (Delta Lake, Unity Catalog, clusters, jobs) are the same on every cloud; the differences are mainly storage, networking and identity integration.

Section B: Compute

Q6. What is a cluster in Databricks?

A cluster is a set of virtual machines that run your code in parallel: one **driver node** and zero or more **worker nodes**. The driver runs your notebook code, builds the Spark execution plan and coordinates work. Workers (executors) run the tasks on partitions of the data.

Today the word "compute" is used in the UI for clusters. You choose the Databricks Runtime version, node types, number of workers (fixed or autoscaling), auto-termination time and access mode.

Q7. What is the difference between an all-purpose cluster and a job cluster?

- **All-purpose cluster**: created manually, shared by many users, used for interactive development and exploration. It stays up until terminated and has a higher DBU price.
- **Job cluster**: created automatically when a job run starts and terminated when the run ends. It is isolated per run, cheaper per DBU, and is the right choice for production pipelines.

Best practice: develop on an all-purpose cluster (or serverless), run production on job clusters or serverless jobs.

Q8. What are autoscaling and auto-termination?

Autoscaling lets a cluster grow and shrink between a minimum and maximum number of workers based on the workload, so you pay for what you need. **Auto-termination** shuts the cluster down after a set period of inactivity (for example 30 or 60 minutes), which prevents idle clusters from burning money. Always set auto-termination on interactive clusters.

Q9. What is the Databricks Runtime (DBR)? What is an LTS version?

The Databricks Runtime is the image that runs on cluster nodes. It contains Apache Spark, Delta Lake, Python, Scala, Java, R, common libraries and Databricks performance improvements. Variants include the standard runtime, **Databricks Runtime ML** (adds ML libraries and GPU support) and Photon-enabled runtimes.

LTS means Long-Term Support: a release that gets bug and security fixes for an extended period (typically about 2 to 3 years), so production jobs should pin an LTS version instead of "latest".

Q10. What is Photon?

Photon is Databricks' native, vectorized query engine written in C++. It runs Spark SQL and DataFrame operations faster than the JVM engine, especially for scans, joins, aggregations and Delta writes. It is Spark API compatible, so you do not change code, you just enable it on the cluster (it is on by default for SQL warehouses and serverless). It costs a higher DBU rate but often finishes much sooner, so the total cost can be lower.

Q11. What is a SQL warehouse?

A SQL warehouse is compute optimised for SQL workloads: the SQL editor, dashboards, BI tools (Power BI, Tableau) and JDBC/ODBC connections. Types are **Serverless** (fast start in seconds, fully managed, recommended), **Pro** and **Classic**. You choose a T-shirt size (2X-Small up to 4X-Large) and optional autoscaling by number of clusters for concurrency.

Q12. What is serverless compute?

With serverless compute Databricks provisions and manages the infrastructure for you. There is no cluster to configure, startup takes seconds, it scales automatically and you pay only for what you use. It is available for notebooks, jobs, pipelines and SQL warehouses. The trade-offs are less control over instance types and some library or networking limitations compared with classic clusters.

Section C: Notebooks and Developer Basics

Q13. What are Databricks notebooks and magic commands?

A notebook is a web document with cells of code and markdown that can mix languages. The default language is set per notebook, and a cell can override it with a magic command:

- `%python`, `%sql`, `%scala`, `%r`: run that language in the cell.
- `%md`: markdown documentation.
- `%sh`: shell commands on the driver.
- `%run`: run another notebook inline.
- `%pip install`: install notebook-scoped libraries.

Notebooks can be scheduled as jobs, versioned with Git folders and shared with permissions.

Q14. What is dbutils? Give common examples.

`dbutils` is a built-in utility library for notebook tasks.

```
dbutils.fs.ls("/Volumes/main/raw/landing/")        # list files
dbutils.fs.cp(src, dst, recurse=True)              # copy files
dbutils.secrets.get(scope="kv", key="db-password") # read a secret
dbutils.widgets.text("run_date", "2026-01-01")     # create a parameter
dbutils.widgets.get("run_date")                    # read a parameter
dbutils.notebook.run("child_nb", 600, {"x": "1"})  # call a notebook
dbutils.notebook.exit("done")                      # return a value
```

Secret values are redacted in notebook output, so they never appear in plain text.

Q15. What is the difference between %run and dbutils.notebook.run()?

- `%run ./other_notebook` includes the other notebook *inline* in the same session, so its functions and variables become available in the calling notebook. It cannot take dynamic parameters or return values, and it is best for shared helper functions.
- `dbutils.notebook.run(path, timeout, params)` starts the other notebook as a separate, *ephemeral job run*, passes parameters through widgets and returns a string set with `dbutils.notebook.exit()`. It can be used in loops and with try/except for retry logic, but variables are not shared.

Q16. What is DBFS? What are Unity Catalog Volumes?

DBFS (Databricks File System) is a file-system abstraction over cloud storage with a workspace-level root. It is legacy for new work: storing data in the DBFS root and using mounts is no longer recommended because it bypasses Unity Catalog governance.

Volumes are Unity Catalog objects for governed access to non-tabular files (CSV, JSON, images, PDFs, ML artifacts). They are addressed as `/Volumes/<catalog>/<schema>/<volume>/path`. A *managed* volume stores files in the schema's managed location; an *external* volume points to a path in your own storage. Use Volumes for files and tables for structured data.

Q17. How do you read a CSV file into a DataFrame and write it as a Delta table?

```
df = (spark.read
      .format("csv")
      .option("header", "true")
      .option("inferSchema", "true")
      .load("/Volumes/main/raw/landing/sales/"))

(df.write
   .format("delta")
   .mode("overwrite")
   .saveAsTable("main.bronze.sales"))
```

In production, avoid `inferSchema` on large files because it scans the data twice; supply an explicit schema instead. Write modes are `append`, `overwrite`, `ignore` and `errorifexists` (the default).

Section D: Delta Lake Basics

Q18. What is Delta Lake?

Delta Lake is an open-source storage layer that sits on top of Parquet files in cloud storage and turns them into reliable tables. It adds a transaction log, so you get ACID transactions, scalable metadata, schema enforcement and evolution, time travel (versioning), efficient upserts (MERGE), and unified batch and streaming reads and writes. In Databricks, Delta is the default table format.

Q19. What are the key features of Delta Lake?

- **ACID transactions**: readers never see partial writes, and failed jobs do not corrupt the table.
- **Schema enforcement**: writes that do not match the table schema are rejected.
- **Schema evolution**: new columns can be added intentionally.
- **Time travel**: query or restore older versions.
- **DML support**: UPDATE, DELETE and MERGE on a data lake.
- **Unified batch and streaming**: a table can be both a streaming source and sink.
- **Performance features**: OPTIMIZE, data skipping, Z-ordering and liquid clustering.
- **Audit history**: `DESCRIBE HISTORY` shows every operation.

Q20. What is the difference between Parquet and Delta?

Delta data files *are* Parquet files, so Delta is not a different data format for the rows. The difference is the extra **transaction log** (the `_delta_log` folder). Plain Parquet is just a folder of files: no ACID guarantees, no UPDATE/DELETE/MERGE, no time travel, no schema enforcement and no concurrent safe writes. Delta = Parquet + transaction log + features built on that log.

Q21. What is the Delta transaction log?

It is the `_delta_log` directory inside every Delta table. Each commit writes a new numbered JSON file (for example `00000000000000000012.json`) recording which data files were added or removed, plus metadata and schema changes. To read the table, Spark replays the log (starting from the latest checkpoint Parquet file, written periodically, by default every 10 commits) to know exactly which files make up the current version. The log is the single source of truth, and it is what makes ACID and time travel possible.

Q22. What is time travel in Delta Lake? How do you use it?

Because old data files are retained until VACUUM removes them, you can read a table as it was at an earlier version or timestamp.

```
SELECT * FROM main.silver.orders VERSION AS OF 12;
SELECT * FROM main.silver.orders TIMESTAMP AS OF '2026-10-01';
DESCRIBE HISTORY main.silver.orders;
RESTORE TABLE main.silver.orders TO VERSION AS OF 12;
```

Use cases: auditing, reproducing an ML training set, comparing before and after a pipeline change, and rolling back a bad write. Time travel is limited by the retention settings (log retention 30 days and VACUUM retention 7 days by default).

Q23. What is the difference between managed and external tables?

- **Managed table**: Unity Catalog manages both metadata and the data files, stored in the managed location of the metastore, catalog or schema. `DROP TABLE` removes the data (kept recoverable for a period, with UNDROP available in Unity Catalog). Recommended default.
- **External table**: you specify a `LOCATION` in your own storage (through an external location). `DROP TABLE` removes only the metadata, and the files stay. Use it when other systems own the path or the data must stay in a specific location.

Q24. How do you update, delete and merge data in a Delta table?

```
UPDATE main.silver.customers SET status = 'inactive'
WHERE last_login < '2025-01-01';

DELETE FROM main.silver.customers WHERE is_test = true;

MERGE INTO main.silver.customers t
USING updates s
ON t.customer_id = s.customer_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

MERGE is the standard **upsert**: update rows that match, insert rows that do not. Each statement is one atomic commit in the transaction log.

Section E: Apache Spark Basics

Q25. What is the difference between RDD, DataFrame and Dataset?

- **RDD**: the low-level, untyped distributed collection of objects. No schema and no optimizer, so it is rarely used now.
- **DataFrame**: a distributed table with named columns and a schema. It uses the Catalyst optimizer and Tungsten engine, so it is faster and the standard choice in Python, SQL, Scala and R.
- **Dataset**: a typed DataFrame, available only in Scala and Java, which gives compile-time type safety.

In PySpark, you use DataFrames.

Q26. What are transformations and actions? What is lazy evaluation?

Transformations (`select`, `filter`, `join`, `groupBy`, `withColumn`) describe a new DataFrame but do not run anything. **Actions** (`count`, `show`, `collect`, `write`, `take`) trigger actual execution.

Spark is **lazy**: it records transformations as a logical plan, optimizes the whole plan, and only runs it when an action is called. This lets Spark push filters down, prune columns and combine steps. A common mistake is thinking a line of code ran when it only built a plan.

Q27. What is the difference between narrow and wide transformations?

A **narrow** transformation (`filter`, `select`, `map`) needs data only from one input partition, so no data movement is needed. A **wide** transformation (`groupBy`, `join`, `distinct`, `orderBy`) needs data from many partitions, which causes a **shuffle**: data is written to disk and moved across the network. Shuffles are the most expensive operations in Spark, and most performance tuning is about reducing or optimising them.

Q28. What are the common DataFrame operations you use daily?

```
from pyspark.sql import functions as F
```

result = (orders .filter(F.col("status") == "COMPLETED") .withColumn("net", F.col("amount") - F.col("discount")) .groupBy("customer_id") .agg(F.sum("net").alias("total_net"), F.count("*").alias("orders")) .orderBy(F.desc("total_net")))

Others: `select`, `drop`, `distinct`, `dropDuplicates`, `withColumnRenamed`, `join`, `union`, `fillna`, `when/otherwise`, `explode` and window functions.

Q29. What are the types of joins in Spark?

- **inner**: only matching keys from both sides.
- **left (left outer)**: all rows from the left, matches from the right, NULL otherwise.
- **right (right outer)**: the mirror of left.
- **full outer**: all rows from both sides.
- **left semi**: left rows that have a match, returning only left columns (like EXISTS).
- **left anti**: left rows that have no match (like NOT EXISTS).
- **cross**: Cartesian product, to be used very carefully.

Example: `orders.join(customers, "customer_id", "left")`.

Q30. What is a temporary view and what is a global temporary view?

`df.createOrReplaceTempView("v")` registers a DataFrame as a SQL-queryable name that lives only in the current Spark session (for example, one notebook). A **global temp view** (`createGlobalTempView`) is visible across sessions of the same application through the `global_temp` schema. For anything shared or permanent, create a real view or table in Unity Catalog instead.

Section F: Governance, Orchestration and Good Practices

Q31. What is Unity Catalog? What is the three-level namespace?

Unity Catalog (UC) is Databricks' unified governance solution for data and AI assets, centrally managed at the account level. It provides fine-grained access control, auditing, lineage, data discovery and sharing across all workspaces.

Objects are addressed as **catalog.schema.table**. The hierarchy is: **metastore** (one per region) then **catalogs** then **schemas** then tables, views, volumes, functions and ML models. Example: `main.sales.orders`. Permissions are granted with SQL such as `GRANT SELECT ON TABLE main.sales.orders TO analysts`.

Q32. What is the Medallion architecture?

A layered design for organising Lakehouse data by quality:

- **Bronze**: raw data exactly as ingested, append-only, with metadata such as load time and source file. Keeps history and allows replay.
- **Silver**: cleaned, deduplicated, typed and conformed data, with joins and business rules applied.
- **Gold**: business-level aggregates and star-schema tables ready for BI, reporting and ML features.

Benefits: clear quality progression, easy reprocessing from Bronze, and different access rules per layer.

Q33. What are Databricks Jobs (Lakeflow Jobs)?

Jobs is the built-in orchestrator, formerly called Databricks Workflows and now part of Lakeflow. A job has one or more **tasks** (notebook, Python script, SQL, JAR, dbt, pipeline, and so on) with dependencies between them, forming a DAG. It supports schedules (cron), file-arrival and table-update triggers, continuous mode, parameters, retries, timeouts, email or webhook alerts, and **Repair run** to re-run only failed tasks.

Q34. How do you manage secrets such as passwords and API keys?

Never hard-code credentials in notebooks. Store them in a **secret scope** (Databricks-backed, or backed by Azure Key Vault) and read them with `dbutils.secrets.get(scope, key)`. Access to scopes is controlled by ACLs, and secret values are redacted in output. For service-to-service access prefer service principals with OAuth, or Unity Catalog storage credentials, instead of long-lived keys.

Q35. How does Git integration work in Databricks?

Git folders (formerly Repos) connect a workspace folder to a remote repo on GitHub, GitLab, Azure DevOps or Bitbucket. You can clone, create branches, commit, push, pull and resolve conflicts from the UI. Notebooks are stored as source files. A typical flow: develop on a feature branch in a personal Git folder, open a pull request, and let CI/CD deploy the merged code to higher environments.

Q36. What are some basic best practices for a beginner on Databricks?

- Use Delta tables and Unity Catalog by default.
- Set auto-termination and right-size clusters; prefer serverless or job compute for production.
- Provide explicit schemas for ingestion.
- Do not use `collect()` on large data; it pulls everything to the driver.
- Keep code in Git and parameterise notebooks with widgets.
- Store secrets in secret scopes.
- Use the Medallion layers and name objects consistently.

Level 2: MEDIUM (Practitioner)

Hands-on working knowledge for 2 to 5 years of experience: Delta in depth, ingestion and streaming, pipelines, Unity Catalog, Spark performance, cost and CI/CD.

Section A: Delta Lake in Depth

Q1. How does Delta Lake provide ACID transactions on cloud object storage?

Object storage has no native transactions, so Delta uses the **transaction log** plus **optimistic concurrency control**.

- A writer first writes new Parquet data files. Nothing is visible yet.
- It then tries to commit by atomically creating the next numbered JSON file in `_delta_log` (for example version 13) that lists the files added and removed.
- Only one writer can create a given version file. If two writers race, one wins, and the other checks whether its changes conflict with what was just committed. If they do not conflict, it retries on the next version; if they do, it fails with a concurrency exception.
- Readers always read a consistent snapshot defined by the log, so they never see half-written data.

Atomicity = the commit file either exists or it does not. **Isolation** = snapshot reads and conflict checking. **Durability** = data lives in cloud storage.

Q2. What do OPTIMIZE and ZORDER do? What is the small-files problem?

Streaming and frequent small appends create many tiny files. Each file adds overhead for listing, opening and task scheduling, so queries slow down. This is the **small-files problem**.

`OPTIMIZE table` compacts small files into larger ones (bin-packing, target up to about 1 GB per file). `OPTIMIZE table ZORDER BY (col1, col2)` also co-locates rows with similar values of those columns in the same files, which improves **data skipping** for queries filtering on them.

```
OPTIMIZE main.silver.orders ZORDER BY (customer_id);
```

Compaction creates new files and marks old ones as removed in the log; the old files are physically deleted later by VACUUM. For new tables, **liquid clustering** is preferred over ZORDER.

Q3. What does VACUUM do, and what are the risks?

`VACUUM` permanently deletes data files that are no longer referenced by the current table version and are older than the retention threshold (default **7 days / 168 hours**). It saves storage cost.

Risks and rules:

- After VACUUM, you cannot time travel to versions that need the deleted files.
- Setting retention below 7 days is dangerous: a long-running query or a concurrent stream may still be reading old files and will fail. Databricks blocks it unless you disable the retention safety check, so avoid that.
- VACUUM does not touch the `_delta_log` JSON; log history has its own retention (`delta.logRetentionDuration`, default 30 days).

```
VACUUM main.silver.orders RETAIN 168 HOURS;
```

Q4. What is the difference between schema enforcement and schema evolution?

Schema enforcement (schema-on-write validation) is the default: Delta rejects a write if columns do not exist in the table, data types are incompatible or a constraint is violated. This protects data quality.

Schema evolution lets the schema change on purpose:

- Append with new columns: `df.write.option("mergeSchema", "true")`.
- Replace the schema on overwrite: `.option("overwriteSchema", "true")`.
- For MERGE, enable `spark.databricks.delta.schema.autoMerge.enabled`, or use `MERGE WITH SCHEMA EVOLUTION`.
- `ALTER TABLE ... ADD COLUMNS` for explicit changes.

Good practice: enforce strictly in Silver and Gold, and allow controlled evolution in Bronze, where the source can change.

Q5. Explain MERGE INTO. What are its common problems and performance tips?

MERGE compares a source to a target on a key and applies matched, not-matched and not-matched-by-source actions in one atomic operation.

```
MERGE INTO main.silver.customers t
USING staged_updates s
ON t.customer_id = s.customer_id
WHEN MATCHED AND s.op = 'D' THEN DELETE
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

Common problems:

- **Duplicate source keys**: if multiple source rows match one target row, MERGE fails with an error. Deduplicate the source first (for example with `row_number()` ordered by update time).
- **Full-table scans**: add the partition or clustering column to the ON condition so Delta can prune files.
- **Many small updates**: batch them; consider deletion vectors to cut the cost of rewriting files.
- Concurrent MERGEs on the same files cause conflicts; partition or cluster so writers touch disjoint data.

Q6. When do you use partitioning, Z-order or liquid clustering?

- **Partitioning** (`PARTITIONED BY`) physically splits data into folders. Use it only for very large tables (rule of thumb: over 1 TB) and on low-cardinality columns such as date, so each partition holds at least about 1 GB. Over-partitioning creates tiny files and hurts performance. You cannot change partition columns without rewriting the table.
- **Z-ordering** clusters data by one or more columns during OPTIMIZE. It is useful on existing tables, but it is not incremental and you must re-run it.
- **Liquid clustering** (`CLUSTER BY (cols)`) is the modern replacement for both. You can change clustering keys without rewriting the table, it clusters incrementally, handles skew and high-cardinality columns, and it is recommended for new Delta tables.

Q7. What is Change Data Feed (CDF) in Delta?

CDF records row-level changes between versions of a Delta table so downstream consumers can process only what changed. Enable it with the table property `delta.enableChangeDataFeed = true`.

```
spark.read.format("delta") \
    .option("readChangeFeed", "true") \
    .option("startingVersion", 5) \
    .table("main.silver.customers")
```

Each row carries `_change_type` (`insert`, `update_preimage`, `update_postimage`, `delete`), `_commit_version` and `_commit_timestamp`. It is used for incremental Silver to Gold loads, audit trails and sending changes to other systems. It only captures changes after it is enabled.

Q8. What are SHALLOW CLONE, DEEP CLONE and RESTORE?

- **DEEP CLONE** copies metadata and data files, creating an independent copy. Good for backups, migration and disaster recovery, and it can be refreshed incrementally.
- **SHALLOW CLONE** copies only metadata and points to the source's data files. It is instant and cheap, which makes it ideal for testing or experimenting on production-like data without copying it. If the source is vacuumed, the clone's references can break.
- **RESTORE** rolls the same table back to an earlier version (`RESTORE TABLE t TO VERSION AS OF 5`), recorded as a new commit.

Section B: Ingestion and Structured Streaming

Q9. What is Auto Loader and why use it?

Auto Loader (`cloudFiles` source) incrementally and efficiently ingests new files as they arrive in cloud storage, without reprocessing old files.

```
(spark.readStream.format("cloudFiles")
   .option("cloudFiles.format", "json")
   .option("cloudFiles.schemaLocation", "/Volumes/main/raw/_schemas/ev")
   .load("/Volumes/main/raw/landing/events/")
 .writeStream
   .option("checkpointLocation", "/Volumes/main/raw/_chk/ev")
   .trigger(availableNow=True)
   .toTable("main.bronze.events"))
```

Key points: it tracks processed files in the checkpoint (exactly-once for file ingestion), scales to millions of files, supports **schema inference and evolution**, and puts unparseable data in a `_rescued_data` column. File discovery uses either **directory listing** (default, simple) or **file notification** (event-based, better at huge scale).

Q10. What is the difference between Auto Loader and COPY INTO?

Both load files idempotently into Delta tables.

- **COPY INTO**: a SQL command, simple, retriable, good for thousands of files, batch-style and scheduled.
- **Auto Loader**: a streaming source built for scale (millions of files), with better schema evolution, checkpoints, flexible triggers and lower file-listing cost.

Databricks recommends Auto Loader for most ingestion, and COPY INTO for small, simple, SQL-only cases.

Q11. Explain Structured Streaming: sources, sinks, triggers, checkpoints and output modes.

Structured Streaming treats a stream as an unbounded table that is processed incrementally using the same DataFrame API as batch.

- **Sources**: Kafka, Event Hubs, Kinesis, files (Auto Loader), Delta tables.
- **Sinks**: Delta tables (most common), Kafka, `foreachBatch` for custom logic.
- **Triggers**: default (micro-batches back to back), `processingTime="1 minute"`, and `availableNow=True` (process everything available, then stop, which is ideal for scheduled incremental jobs).
- **Checkpoint**: stores offsets and state so the query resumes where it stopped after a failure. Each query needs its own unique checkpoint location.
- **Output modes**: `append` (only new rows), `update` (changed rows), `complete` (the whole result table, for aggregations).

Q12. What are watermarks and how do they handle late data?

In streaming aggregations over event time, Spark must keep state for each window. A **watermark** tells Spark how long to wait for late events.

```
(events
  .withWatermark("event_time", "10 minutes")
  .groupBy(F.window("event_time", "5 minutes"), "device")
  .count())
```

With a 10-minute watermark, events arriving more than 10 minutes behind the latest seen event time are dropped, and state for old windows is cleaned up. Without a watermark, state grows forever. The trade-off is lateness tolerance against state size and latency.

Section C: Pipelines and Orchestration

Q13. What are Lakeflow Declarative Pipelines (formerly Delta Live Tables)?

A declarative framework for building reliable ETL pipelines. You declare *what* each dataset is (a query), and Databricks handles *how*: dependency ordering, orchestration, retries, autoscaling, checkpoints and monitoring. The naming has changed over time (Delta Live Tables or DLT, then Lakeflow Declarative Pipelines), so know both names.

Dataset types:

- **Streaming table**: incrementally processes append-only sources (ingestion, Bronze and Silver).
- **Materialized view**: a precomputed result that is incrementally refreshed where possible (aggregations, Gold).
- **View**: a temporary intermediate query, not published.

Built-in extras: data quality **expectations**, CDC with AUTO CDC (formerly APPLY CHANGES), event log and lineage.

Q14. What are expectations (data quality rules) in declarative pipelines?

Expectations are constraints on rows, each with a violation action:

- **EXPECT** (warn): keep the row and record the violation in metrics.
- **ON VIOLATION DROP ROW**: drop invalid rows.
- **ON VIOLATION FAIL UPDATE**: stop the pipeline update.

```
CREATE OR REFRESH STREAMING TABLE silver_orders (
  CONSTRAINT valid_id EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT pos_amt  EXPECT (amount > 0)
) AS SELECT * FROM STREAM(bronze_orders);
```

Violation counts appear in the pipeline event log and UI, which gives you data quality monitoring for free.

Q15. How do you design a robust multi-task job in Databricks?

- Break the work into tasks (ingest, transform, test, publish) with explicit dependencies.
- Use **job clusters or serverless**, not all-purpose clusters.
- Pass parameters with job parameters and widgets; share small values between tasks with **task values** (`dbutils.jobs.taskValues`).
- Configure **retries** with sensible limits, **timeouts** and failure notifications.
- Use **Run-if** conditions and if/else or For-each tasks for branching and looping.
- Use **Repair run** to re-run only failed and downstream tasks.
- Make every task idempotent so reruns are safe.
- Define jobs as code with Databricks Asset Bundles.

Section D: Unity Catalog and Governance

Q16. How does access control work in Unity Catalog?

Privileges are granted to users, groups or service principals on securables and are inherited down the hierarchy. A user needs `USE CATALOG` and `USE SCHEMA` on parents plus `SELECT` (or `MODIFY`, `CREATE TABLE`, and so on) on the object.

```
GRANT USE CATALOG ON CATALOG main TO `analysts`;
GRANT USE SCHEMA  ON SCHEMA main.sales TO `analysts`;
GRANT SELECT ON TABLE main.sales.orders TO `analysts`;
```

Best practice: grant to **groups** (synced from your identity provider), not individuals; give ownership to groups; and use views, row filters and column masks for finer control. Every access is logged in audit logs.

Q17. What are storage credentials and external locations?

They let Unity Catalog govern access to your cloud storage without distributing keys.

- A **storage credential** wraps a cloud identity (an IAM role on AWS, a managed identity or service principal on Azure) that can access storage.
- An **external location** links a storage path (for example `abfss://data@account.dfs.core.windows.net/raw`) to a storage credential, and privileges such as `READ FILES` or `CREATE EXTERNAL TABLE` are granted on it.

This replaces mounts and per-cluster credentials and keeps all access auditable.

Q18. What are Unity Catalog lineage and system tables?

Lineage is captured automatically at table and column level for queries run on Databricks, so you can see upstream sources and downstream consumers of any table (also for notebooks, jobs and dashboards). It is very useful for impact analysis and audits.

System tables (the `system` catalog) are read-only, Databricks-managed tables you can query with SQL, for example `system.access.audit` (audit logs), `system.billing.usage` (cost), `system.lineage.*`, and compute and query history tables. They are the basis for governance and cost dashboards.

Q19. What is the Hive metastore, and how do you migrate to Unity Catalog?

The legacy Hive metastore is a per-workspace metastore with two-level naming (`hive_metastore.schema.table`) and limited governance. Unity Catalog is account-level, three-level and fully governed.

Migration approach: assess with a tool such as UCX (Databricks Labs), set up the metastore, storage credentials and external locations, upgrade *managed* tables by copying (`DEEP CLONE` or `CREATE TABLE AS SELECT`), upgrade *external* tables in place with `SYNC`, convert views and permissions, then update jobs, notebooks and code references to the new three-level names, and validate before turning off the old tables.

Section E: Spark Performance and Internals

Q20. Explain how Spark executes a job: driver, executors, jobs, stages and tasks.

The **driver** turns your code into a logical plan, optimizes it (Catalyst), creates a physical plan and a DAG, and schedules work. **Executors** run **tasks** and hold cached data.

Hierarchy: each **action** creates a **job**; a job is split into **stages** at shuffle boundaries; each stage has one **task per partition**. Tasks of a stage run in parallel across the executor cores. In the Spark UI you inspect jobs, stages, task durations, shuffle read/write, spills and the SQL plan.

Q21. What is the difference between repartition() and coalesce()?

- `repartition(n)` does a **full shuffle** and can increase or decrease partitions, producing evenly sized ones. It can also repartition by columns: `repartition(200, "customer_id")`.
- `coalesce(n)` only **decreases** partitions by merging existing ones with no full shuffle. It is cheaper, but the result can be unevenly sized.

Use `coalesce` to reduce the number of output files cheaply, and `repartition` when you need balance or a new key distribution.

Q22. What is a broadcast join and when should you use it?

In a broadcast join, the small table is sent to every executor and joined locally against each partition of the big table, so the big table is **not shuffled**. Spark does this automatically when one side is smaller than `spark.sql.autoBroadcastJoinThreshold` (default 10 MB), and AQE can also convert joins at runtime. You can force it:

```
from pyspark.sql.functions import broadcast
fact.join(broadcast(dim_country), "country_id")
```

Do not broadcast a table that is too large; it can cause driver or executor out-of-memory errors.

Q23. What is the difference between cache() and persist()? What is the Databricks disk cache?

`cache()` is shorthand for `persist()` with the default storage level (`MEMORY_AND_DISK` for DataFrames); `persist(level)` lets you choose the level. Caching is useful when the same DataFrame is used by several actions, such as iterative algorithms. It is lazy: data is cached on the first action. Release with `unpersist()`.

The **disk (IO) cache** is separate: Databricks automatically caches remote Parquet and Delta data on the workers' local SSDs, speeding up repeated reads, with no code change. Avoid caching big DataFrames needlessly, as it wastes memory.

Q24. What is data skew and how does Adaptive Query Execution (AQE) help?

Skew means a few keys hold a disproportionate amount of data, so a few tasks run much longer than the others while the cluster sits idle. You see it in the Spark UI as max task time far above the median.

AQE (on by default) re-optimises the plan at runtime using real statistics. It can **coalesce** small shuffle partitions, **switch** a sort-merge join to a broadcast join and **split skewed partitions** in joins. Other fixes: filter early, salt the skewed key, or broadcast the smaller side.

Q25. Why are Python UDFs slow, and what are the alternatives?

A regular Python UDF forces Spark to serialize rows between the JVM and a Python process, one row at a time, and blocks Catalyst optimisation, and Photon cannot accelerate it. Preference order:

- First use **built-in functions** (`pyspark.sql.functions`) or Spark SQL expressions.
- Then use **pandas (vectorized) UDFs**, which exchange data in Arrow batches and are much faster.
- Use a Python UDF only when there is no alternative, and consider a SQL UDF registered in Unity Catalog for reusable logic.

Q26. Coding question: keep only the latest record per business key.

```
from pyspark.sql import Window, functions as F
```

w = Window.partitionBy("customer_id").orderBy(F.desc("updated_at")) ~ latest = (df .withColumn("rn", F.row_number().over(w)) .filter("rn = 1") .drop("rn"))

`row_number()` guarantees exactly one row per key (use `rank()` or `dense_rank()` if ties must be kept). The same pattern is used to deduplicate the source before a MERGE.

Section F: Operations, Cost and Delivery

Q27. How do cluster policies, pools and access modes help?

- **Cluster policies** restrict and pre-set cluster settings (allowed node types, maximum size, mandatory auto-termination, required tags), which controls cost and enforces standards.
- **Instance pools** keep idle ready-to-use VMs so clusters start faster and cost less for short jobs.
- **Access modes** define isolation and security: **Standard** (shared by multiple users, with Unity Catalog enforcement) and **Dedicated** (assigned to one user or group, needed for some ML and RDD workloads).

Q28. How do you reduce Databricks cost?

- Use job clusters or serverless instead of always-on all-purpose clusters.
- Set auto-termination and autoscaling; right-size nodes using Spark UI evidence.
- Use spot instances for fault-tolerant workloads (keep the driver on-demand).
- Enable Photon where it shortens runtime, and pick the latest runtime.
- Fix the root causes: small files, skew, unnecessary shuffles, repeated full reloads (use incremental processing).
- Apply cluster policies and tags, then monitor `system.billing.usage` and set budgets.
- Let Predictive Optimization maintain tables.

Q29. What are Databricks Asset Bundles (DABs) and how do you do CI/CD?

A bundle is a YAML-defined project (`databricks.yml`) that describes jobs, pipelines, clusters, dashboards and permissions as code, with **targets** such as dev, staging and prod.

Typical flow: develop locally or in a Git folder, push to a feature branch, CI runs unit tests (pytest) and `databricks bundle validate`, a merge to main deploys with `databricks`

`bundle deploy -t prod` using a **service principal**, and `databricks bundle run` triggers integration tests. This gives repeatable, reviewable, environment-consistent deployments.

Q30. What is a service principal and why use it?

A service principal is a non-human identity for automation (CI/CD, production jobs, applications). Jobs should run as a service principal, not a person, so they do not break when someone leaves, permissions are least-privilege and audit logs clearly show automated activity. Authenticate with OAuth machine-to-machine (client ID and secret) rather than personal access tokens where possible.

Q31. How would you lay out a practical Medallion pipeline on Databricks?

- **Landing**: files arrive in a Unity Catalog volume or external location.
- **Bronze**: Auto Loader or a streaming table ingests raw data as is, with `_ingest_time` and `_source_file` columns; schema evolution is permitted.
- **Silver**: deduplicate, cast types, validate with expectations, apply CDC or MERGE, and conform to entities.
- **Gold**: aggregated facts and dimensions, KPIs and feature tables, served by a SQL warehouse to Power BI or dashboards.
- Orchestrate with Jobs, govern with Unity Catalog (catalogs per environment and per layer), and monitor with system tables and alerts.

Q32. What can you do in Databricks SQL (dashboards, alerts, Genie)?

Databricks SQL gives analysts a SQL editor, saved queries, **AI/BI dashboards**, **alerts** that notify when a query result crosses a threshold, scheduling, and connections for BI tools. **Genie** is a natural-language interface where business users ask questions about curated data and get SQL-backed answers. Data stays governed by Unity Catalog permissions.

Q33. What is MLflow and where does it fit?

MLflow is an open-source platform for the machine-learning lifecycle, managed inside Databricks. **Tracking** logs parameters, metrics and artifacts for each run, **Models** packages models in a standard format, and the **Model Registry** (in Unity Catalog) versions models, with aliases such as "champion". Models can then be deployed to batch inference or **Model Serving** endpoints. Even as a data engineer you should know the basics.

Level 3: HIGH (Advanced and Architect)

Internals, design and scenario questions for senior and lead roles: transaction-log internals, concurrency, liquid clustering, advanced streaming, security architecture, DR, migration and production troubleshooting.

Section A: Delta Lake and Engine Internals

Q1. Explain the internals of the Delta transaction log: commits, checkpoints and data skipping.

The `_delta_log` folder holds ordered JSON commit files. Each commit contains **actions**: `metaData` (schema, partition columns, properties), `protocol` (reader and writer versions), `add` (a new data file with its size and **per-file column statistics**: min, max, null count, record count), `remove` (a logically deleted file) and `commitInfo` (operation, user, metrics).

To avoid replaying thousands of JSON files, Delta writes a **checkpoint** Parquet file (by default every 10 commits) holding the full table state at that version, and `_last_checkpoint` points to it. A reader loads the latest checkpoint and replays only the later JSON commits.

Data skipping: the min/max statistics in `add` actions let the engine skip files that cannot contain matching rows. Statistics are collected for the first 32 columns by default (`delta.dataSkippingNumIndexedCols`), so put filter columns early in the schema or tune this property; avoid long strings in the stats columns. Clustering (Z-order or liquid) makes the min/max ranges tighter and skipping more effective.

Q2. How does Delta handle concurrent writes? Explain isolation levels and typical conflicts.

Delta uses optimistic concurrency. At commit time, a transaction checks whether files it read or modified were changed by transactions that committed after it started.

Isolation levels:

- **WriteSerializable** (default): writes are serializable among themselves, but reads can see a state that is not identical to any serial order. It gives higher availability and is enough for most workloads.
- **Serializable** (`delta.isolationLevel`): the strongest level; all operations (including reads) behave as if executed serially. It produces more conflicts.

Conflict exceptions include `ConcurrentAppendException` (another transaction added files in the partition you read), `ConcurrentDeleteReadException`,

`ConcurrentDeleteDeleteException` and `ProtocolChangedException`.

How to reduce conflicts: make the condition in MERGE, UPDATE and DELETE explicit on the partition or clustering column so transactions touch disjoint files; retry on conflict; avoid many writers touching the same files; and use tables with **deletion vectors**, where **row-level concurrency** detects conflicts at row level instead of file level.

Q3. What are deletion vectors, and what is Predictive I/O?

Deletion vectors are a storage optimisation. Normally, an UPDATE, DELETE or MERGE that changes one row rewrites the entire Parquet file. With deletion vectors (`delta.enableDeletionVectors`), Delta marks the affected rows as deleted in a small side file and the physical rewrite is deferred until later (for example during OPTIMIZE or `REORG TABLE ... APPLY (PURGE)`). Writes become much faster, and row-level concurrency becomes possible. Readers filter out marked rows. The Photon engine reads them efficiently, and they are enabled by default on new tables in recent runtimes.

Predictive I/O is a set of Databricks read and write accelerations for Photon (including faster selective reads and faster MERGE/UPDATE/DELETE) that reduce the need for manual indexing or tuning.

Q4. Explain liquid clustering in depth. How does it compare to partitioning and Z-order, and what are its limits?

Liquid clustering is declared with `CLUSTER BY (col1, col2)` at table creation, or added later with `ALTER TABLE ... CLUSTER BY`. It replaces both partitioning and ZORDER for new tables.

Advantages:

- **Flexible**: you can change the clustering keys at any time without rewriting existing data; old files stay laid out under the old keys until they are re-clustered.
- **Incremental**: `OPTIMIZE` only reclusters new or not-yet-clustered data, so cost is lower than repeated full Z-ordering.
- **Handles skew and high cardinality**, which break partitioning, and avoids partition-count explosion.
- **Automatic liquid clustering** (`CLUSTER BY AUTO`) with Predictive Optimization lets Databricks choose and evolve keys from the observed query patterns.

Limits and notes: a small, limited number of clustering columns (up to four); the columns must have statistics collected; it is not combined with partitioning or ZORDER on the same table; and clustering only happens when OPTIMIZE (or Predictive Optimization or the clustering-on-write path) runs, so scheduling it matters.

Q5. What is Predictive Optimization?

Predictive Optimization is a Unity Catalog feature that automatically runs maintenance on **managed tables**: **OPTIMIZE** (compaction and clustering), **VACUUM** (cleanup of unreferenced files) and **ANALYZE** (statistics collection), choosing when each is worthwhile and running them on serverless compute. It removes the need to schedule maintenance jobs yourself, reduces storage and query cost, and its activity is visible in system tables. It can be enabled at account, catalog or schema level.

Q6. Explain the Catalyst optimizer, Tungsten and the key AQE features.

Catalyst turns a query into an optimized plan in four phases: analysis (resolve columns and types against the catalog), logical optimisation (predicate pushdown, column pruning, constant folding, join reordering), physical planning (choose strategies, for example the join algorithm, using cost-based statistics) and code generation.

Tungsten improves execution: off-heap binary memory, cache-friendly data layout and whole-stage code generation, which compiles a pipeline of operators into one tight loop. **Photon** goes further with a native vectorized engine.

Adaptive Query Execution re-plans at shuffle boundaries using actual runtime statistics: it coalesces small shuffle partitions, converts sort-merge joins into broadcast joins when one side turns out to be small, and splits skewed partitions in joins (by default a partition is treated as skewed when it is over 5 times the median size and over 256 MB). Because of AQE, you can often set `spark.sql.shuffle.partitions` high and let Spark coalesce.

Q7. A job fails with out-of-memory errors or heavy spill. How do you diagnose and fix it?

First identify *where* memory is exhausted.

- **Driver OOM**: caused by `collect()`, `toPandas()` on large data, or a huge broadcast. Fix by keeping data distributed, writing to storage instead of collecting, lowering the broadcast threshold, or using a larger driver.
- **Executor OOM or spill**: caused by large or skewed partitions, wide aggregations, or too few shuffle partitions. In the Spark UI (Stages tab), look at **Spill (memory/disk)**, max versus median task size and GC time.

Fixes, in order: remove unnecessary data early (filter, select only needed columns); fix skew (AQE, salting); increase the number of shuffle partitions so each partition is smaller; use memory-optimised nodes if the workload really needs more memory per core; avoid caching what you do not reuse; replace Python UDFs; and only then scale up. Adding nodes does not fix skew.

Section B: Performance Scenarios

Q8. A pipeline that used to take 30 minutes now takes 3 hours. How do you troubleshoot it systematically?

I would follow a structured approach rather than guess.

- **What changed?** Compare data volume, a new source, code changes, runtime version, cluster size or concurrent workloads. Check the job run history for the trend.
- **Find the slow stage**: Spark UI, SQL/DataFrame tab for the expensive operator, and the Stages tab for the longest stage.
- **Classify the symptom**: one huge task means skew; lots of spill means memory pressure or partitions that are too large; thousands of tiny tasks means small files or too many partitions; long scan time means missing data skipping, small files or no pruning; a long shuffle means a bad join strategy; long gaps between jobs mean driver-side work or a slow cloud API.
- **Check the data layout**: file count and size (`DESCRIBE DETAIL`), whether OPTIMIZE has run, partition or clustering choice.
- **Apply the targeted fix** (broadcast, salting, repartition, compaction, clustering, incremental logic, Photon), then re-measure.
- **Prevent regression**: add monitoring, alerts on duration, and cost tags.

Q9. How do you handle a skewed join using salting?

When AQE skew handling is not enough (for example in aggregations or an extreme hot key), add a random **salt** to spread a hot key across many partitions.

```
N = 16
big = big.withColumn("salt", (F.rand() * N).cast("int"))
small = small.withColumn("salt", F.explode(F.array(
          *[F.lit(i) for i in range(N)])))
joined = big.join(small, ["key", "salt"])
```

The large side gets a random salt in 0..N-1, and the small side is replicated N times with every salt value so every salted key still finds a match. For aggregations, do a two-stage aggregation: first on (key, salt), then on key. Also consider isolating the hot keys, processing them separately with a broadcast join, and unioning the results.

Q10. Explain the Spark join strategies and how Spark chooses.

- **Broadcast Hash Join**: one side small enough to broadcast; no shuffle of the big side; the fastest, for equi-joins.
- **Shuffle Hash Join**: both sides shuffled by key and a hash table built on the smaller side per partition; needs that side to fit in memory.
- **Sort-Merge Join**: both sides shuffled and sorted by key, then merged; the default for large equi-joins, scalable and robust, and can spill to disk.
- **Broadcast Nested Loop Join** and **Cartesian Join**: for non-equi conditions or cross joins; very expensive.

Spark picks using statistics (sizes versus `autoBroadcastJoinThreshold`) and join hints (`BROADCAST`, `MERGE`, `SHUFFLE_HASH`, `SHUFFLE_REPLICATE_NL`). AQE can change the strategy at runtime. Up-to-date statistics (`ANALYZE TABLE`) and data layout help the planner choose well.

Q11. How do you handle file sizing and the small-files problem at scale?

Aim for files of roughly 128 MB to 1 GB. Techniques:

- **Optimized writes** (`delta.autoOptimize.optimizeWrite`): shuffle before write so each task writes fewer, larger files.
- **Auto compaction** (`delta.autoOptimize.autoCompact`): small compaction runs after writes.
- Scheduled `OPTIMIZE`, or Predictive Optimization for managed tables.
- Do not over-partition; prefer liquid clustering.
- Avoid very frequent tiny streaming micro-batches into a table: use a longer trigger interval, or `availableNow` batches.
- For file-count control at write time, `coalesce` or `repartition` before writing.

Remember that VACUUM is needed afterwards to physically remove compacted-away files.

Q12. How do you tune the number of shuffle partitions?

`spark.sql.shuffle.partitions` defaults to 200 and controls how many partitions a shuffle produces. Too few creates large partitions, spill and OOM; too many creates tiny tasks with scheduling overhead. A good target is partitions of about 100 to 200 MB after the shuffle, with at least a few times as many tasks as total cores. On Databricks, `auto` shuffle partition settings and AQE coalescing can pick a good number at runtime, so a common approach is to leave AQE on and set the number high enough for the largest stage. Verify using the Spark UI shuffle read size per task.

Section C: Advanced Streaming and Ingestion

Q13. How do you achieve exactly-once processing in Structured Streaming?

Exactly-once end-to-end needs three things: a **replayable source** (Kafka offsets, Delta versions or Auto Loader file tracking), **checkpointing** of offsets and state, and an **idempotent or transactional sink**. Delta Lake sinks are transactional, and the stream records the batch ID in the Delta transaction, so a retried micro-batch is not written twice.

For custom logic with `foreachBatch`, the batch can be retried, so the write must be idempotent: use MERGE keyed on a business key, or use the `txnAppId` and `txnVersion` write options to deduplicate. Never change the logic of a running query in an incompatible way (for example the aggregation keys or the source) without a new checkpoint; schema-compatible changes are allowed, but know the documented restrictions.

```
def upsert(batch_df, batch_id):
    batch_df.createOrReplaceTempView("updates")
    batch_df.sparkSession.sql(
      "MERGE INTO main.silver.t USING updates u "
      "ON t.id = u.id WHEN MATCHED THEN UPDATE SET * "
      "WHEN NOT MATCHED THEN INSERT *")

stream.writeStream.foreachBatch(upsert) \
  .option("checkpointLocation", chk).start()
```

Q14. Explain stateful streaming: deduplication, stream-stream joins and state store management.

Stateful operations (aggregations, `dropDuplicates`, stream-stream joins, `flatMapGroupsWithState`) keep **state** between micro-batches, stored in a state store and checkpointed.

- **Deduplication**: `dropDuplicatesWithinWatermark` or `dropDuplicates` on id plus event time with a watermark, so state is bounded.

- **Stream-stream joins**: both sides are buffered; you need a watermark on each side and a time-range join condition so Spark can drop old state.
- **State store**: the default in-memory provider can create GC pressure on large state, so enable the **RocksDB** state store provider (`spark.sql.streaming.stateStore.providerClass`, Databricks has a built-in option) which keeps state on local disk, with changelog checkpointing for faster commits.

Operations to watch: state size and rows per operator in the streaming query progress metrics, watermark delay, and processing versus input rate.

Q15. How do you run Auto Loader reliably at scale (schema drift, backfill, notifications)?

- **Schema drift**: set `cloudFiles.schemaLocation` and choose `cloudFiles.schemaEvolutionMode`: `addNewColumns` (default; the stream stops once to add the column and picks it up on restart), `rescue` (never evolves; new fields go to `_rescued_data`), `failOnNewColumns`, or `none`. Use `schemaHints` to pin types for key columns and avoid wrong inference.
- **Bad data**: keep `_rescued_data` and quarantine rows that fail parsing instead of failing the pipeline.
- **Scale**: use file notification (or file events on Unity Catalog external locations) instead of directory listing when you have a very large number of files.
- **Backfill**: for historic data, use `cloudFiles.includeExistingFiles` and consider `maxFilesPerTrigger` or `maxBytesPerTrigger` to control batch size; run a separate backfill job rather than overloading the live stream.
- **Operations**: auto-restart on schema change using a job with retries; one checkpoint per source and target.

Q16. What trigger options exist and how do you choose between continuous and incremental batch?

Options: default micro-batch (as fast as possible), `processingTime` (fixed interval), `availableNow=True` (process everything available in several rate-limited batches and then stop; this replaced the older `once` trigger) and an experimental continuous mode with very low latency.

Choose by the business latency requirement and cost. Needing seconds-level freshness means a continuously running stream. Needing minutes to hours means `availableNow` on a schedule with a job cluster or serverless: you get the same incremental logic and checkpointing, but the compute only runs and bills while processing. Many teams overspend by running always-on streams for data that is only consumed hourly.

Section D: Data Modelling, Pipelines and Governance Design

Q17. How do you implement SCD Type 2 on Databricks?

SCD Type 2 keeps full history by closing the old version of a row and inserting a new one, with `effective_from`, `effective_to` and `is_current` columns.

Preferred, declarative: use AUTO CDC (formerly APPLY CHANGES INTO) in a Lakeflow pipeline.

```
CREATE OR REFRESH STREAMING TABLE dim_customer;

APPLY CHANGES INTO dim_customer
FROM STREAM(bronze_customer_cdc)
KEYS (customer_id)
SEQUENCE BY updated_at
STORED AS SCD TYPE 2;
```

It handles out-of-order events using the sequence column and generates the `__START_AT` and `__END_AT` columns.

Manual approach with MERGE: first stage the changes so that each changed key appears twice (once with a null merge key to insert the new version, once with the real key to close the old version), then `WHEN MATCHED AND changed THEN UPDATE SET is_current=false, effective_to=...` and `WHEN NOT MATCHED THEN INSERT`. Always deduplicate and order the source by the sequence column first.

Q18. Design an end-to-end data platform on Databricks for 50 source systems, batch and streaming, with strict governance. What do you propose?

I would answer in layers.

- **Ingestion**: Lakeflow Connect or managed connectors for SaaS and databases; Auto Loader for files; Kafka or Event Hubs with Structured Streaming for events; Lakehouse Federation for ad-hoc access to systems you do not want to copy.
- **Storage and modelling**: Delta tables in Unity Catalog with Medallion layers. Separate catalogs per environment (dev, test, prod) and clear schemas per domain; managed tables by default, external locations only when required.
- **Processing**: declarative pipelines for most ETL (expectations for quality, AUTO CDC for CDC), with Spark jobs for special cases; incremental by default.
- **Orchestration**: Lakeflow Jobs with dependencies, retries, alerts, and file-arrival or table-update triggers.
- **Serving**: SQL warehouses (serverless) for BI and AI/BI dashboards, Delta Sharing for external consumers, feature tables and Model Serving for ML.
- **Governance and security**: Unity Catalog RBAC with groups from the identity provider, row filters and column masks for PII, tags and lineage, audit via system tables, private networking and customer-managed keys.
- **Delivery**: Asset Bundles plus Terraform, CI/CD with service principals, automated tests.
- **Operations**: monitoring on job SLAs, data freshness and quality, cost dashboards and budgets, and Predictive Optimization.

Q19. How do you build data quality and observability into pipelines?

- **Prevent**: explicit schemas, Delta constraints (`NOT NULL`, `CHECK`), schema enforcement and pipeline expectations at Bronze to Silver.
- **Detect**: freshness, volume, null-rate and distribution checks; Lakehouse Monitoring profiles and drift metrics on tables; reconciliation counts between layers.
- **Contain**: a quarantine table for failed rows, with a reason column, instead of silent drops or total failure.
- **Alert**: job failure and duration notifications, SQL alerts on quality metrics, and routing to on-call channels.
- **Investigate**: lineage to find upstream causes, Delta history and time travel to compare versions, and the pipeline event log.
- **Own**: data contracts with producers and clear ownership per dataset.

Q20. How do you make pipelines idempotent and replayable? How do you backfill safely?

Idempotent means running the same load twice gives the same result. Techniques: MERGE on business keys instead of blind appends; `replaceWhere` to overwrite exactly the affected partition or date range atomically; deterministic logic (no `current_timestamp()` inside business keys); checkpointed incremental sources.

Replayable means you can rebuild downstream from Bronze, so keep Bronze immutable and complete, with source metadata.

Safe backfill: run it as a parameterised job over a date range, write with `replaceWhere` or MERGE, process in bounded chunks to protect the cluster, keep it separate from the live pipeline (or pause it), validate counts against the source, and rely on Delta history so you can `RESTORE` if something goes wrong.

Q21. How do you implement GDPR 'right to be forgotten' on a Delta lake?

Deleting a row with `DELETE` only logically removes it: the old files still contain the data and remain reachable through time travel. A complete erasure needs three steps:

- Run `DELETE FROM table WHERE user_id = ...` (batch many requests into one operation for efficiency).
- If deletion vectors are enabled, run `REORG TABLE table APPLY (PURGE)` so the underlying files are rewritten without the deleted rows.
- Run `VACUUM` after the retention period so old file versions are physically removed, and make sure retention and time-travel policies match your legal deadline.

Also handle downstream copies (Silver, Gold, backups, CDF output, ML features), logs, and use design patterns that reduce risk: pseudonymise or tokenise identifiers, keep PII in a separate table that is easy to delete from, and use column masks so most users never see PII.

Section E: Security, Sharing and Migration

Q22. Go deeper on Unity Catalog: metastore design, row filters, column masks and ABAC.

Design: one metastore per region, attached to many workspaces. Use catalogs as the main isolation boundary (per environment, domain or business unit), schemas for subject areas, managed storage locations per catalog so data lands in separate storage accounts when required, and external locations only where needed.

Fine-grained control:

- **Dynamic views** use `is_account_group_member()` to filter rows or mask columns per user group.
- **Row filters**: a SQL UDF attached to a table that returns true for rows a user may see.
- **Column masks**: a SQL UDF attached to a column that returns masked or real values by identity.
- **ABAC** (attribute-based access control): governed tags, such as `pii=true`, on tables and columns, with policies that automatically apply masks or filters to everything carrying the tag, so you manage rules once rather than per table.

```
ALTER TABLE main.hr.emp
  ALTER COLUMN salary SET MASK main.gov.mask_salary;
```

Complement this with audit logs and lineage for evidence.

Q23. Describe the security architecture of an enterprise Databricks deployment.

- **Network**: deploy the classic compute plane into your own virtual network (VNet injection or customer-managed VPC), use **secure cluster connectivity** (no public IPs on nodes), **Private Link** for front-end (users and APIs) and back-end (control plane to compute) connectivity, IP access lists, and firewall egress rules. Serverless uses network policies and private endpoints.
- **Identity**: SSO through the identity provider, SCIM sync of users and groups to the account, groups for all permissions, **service principals** with OAuth for automation, MFA at the IdP.
- **Data protection**: encryption at rest with customer-managed keys for managed services and storage, TLS in transit, Unity Catalog for access control, secrets in a vault-backed scope.
- **Compliance and monitoring**: audit logs delivered to storage or queried from system tables, compliance security profile for standards such as HIPAA or PCI where needed, and cluster policies to enforce secure configurations.

Q24. What are Delta Sharing and Lakehouse Federation? When do you use each?

Delta Sharing is an open protocol for securely sharing live data with other organisations or teams *without copying it*. The provider creates a **share** (tables, views, volumes, models) and gives recipients access; recipients can be on Databricks (Databricks-to-Databricks, governed by Unity Catalog) or on any platform (open sharing through pandas, Spark, Power BI, Tableau and others). Use it for external consumers, partners and cross-region or cross-cloud sharing.

Lakehouse Federation lets you run governed queries directly on external systems (PostgreSQL, MySQL, SQL Server, Snowflake, BigQuery, Redshift and others) through foreign catalogs in Unity Catalog, without moving the data. Use it for quick access, prototyping and migration phases; ingest the data into Delta when you need performance

and cost control at scale.

Q25. How would you design disaster recovery and multi-region for Databricks?

Define RPO and RTO first, then design for three layers:

- **Data**: use cloud storage replication (geo-redundant or cross-region replication) for storage accounts, or scheduled **DEEP CLONE** and incremental copy to a second region. Delta Sharing can also replicate data to another region.
- **Metadata and code**: keep jobs, pipelines, clusters, policies and permissions defined as code (Asset Bundles and Terraform) so a second workspace can be rebuilt quickly, and keep notebooks in Git. Unity Catalog metadata for a second region is recreated or synchronised by automation.
- **Compute and operations**: a pre-provisioned or quickly deployable secondary workspace (active-passive is most common), tested failover runbooks, DNS or endpoint switching for serving, and regular DR drills.

Be honest in the interview that cross-region data consistency, streaming checkpoints and ongoing writes are the hard parts, so design for replayable pipelines.

Q26. How would you migrate from on-premises Hadoop/Hive, or from another warehouse, to Databricks?

Use a phased approach:

- **Assess**: inventory workloads, data volumes, dependencies, SLAs and code (HiveQL, Spark, Oozie, stored procedures); classify by complexity.
- **Land the data**: move data to cloud storage (bulk transfer tools, then incremental CDC to keep it in sync); convert to Delta (`CONVERT TO DELTA` for existing Parquet).
- **Translate the code**: Spark and Hive SQL mostly work with small changes; use automated converters for warehouse SQL dialects and stored procedures, and re-platform orchestration to Jobs.
- **Govern**: set up Unity Catalog, groups and permissions before users arrive.
- **Validate**: row counts, checksums and business-metric reconciliation, with parallel runs.
- **Cut over** workload by workload (by domain), then decommission the old system, tracking cost and performance gains.

Section F: Operations, Cost, Delivery and Leadership

Q27. How do you govern and monitor cost across many teams?

- **Visibility**: query `system.billing.usage` joined with price tables, and tag everything (cluster policies enforce required tags such as team and project); build dashboards for DBUs by workspace, job, tag and SKU.
- **Guardrails**: cluster policies (limits, auto-termination, allowed instance types), budgets and alerts, and serverless budget policies.
- **Efficiency**: serverless for spiky or interactive workloads, job compute with spot instances for steady batch, Photon where it pays off, incremental pipelines, Predictive Optimization, and removing idle all-purpose clusters.
- **Accountability**: showback or chargeback per team and regular review of the top cost drivers. Optimise the biggest jobs first, since a few jobs usually dominate the bill.

Q28. How do you structure CI/CD, infrastructure as code and testing for Databricks?

- **Infrastructure**: Terraform provisions workspaces, networking, Unity Catalog metastore, catalogs, grants, storage credentials, cluster policies and service principals.
- **Application**: Databricks Asset Bundles deploy jobs, pipelines and dashboards to dev, staging and prod targets.
- **Code structure**: put business logic in Python packages with small testable functions, and keep notebooks thin.
- **Tests**: unit tests with pytest on local Spark or Databricks Connect; integration tests that run the pipeline on a small dataset in a staging catalog; data tests (expectations and reconciliation) in production.
- **Pipeline**: pull request triggers lint, tests and `bundle validate`; merge to main deploys to staging, then a gated promotion to prod using a service principal, with environment-specific config and secrets.

Q29. How do you monitor and alert on production Databricks workloads?

- **Jobs**: failure, duration and "start delayed" notifications; job run and task metrics; retries and Repair run for recovery.
- **Pipelines**: the event log for expectation metrics, row counts and update status; alerts on failed updates.
- **Streaming**: query progress metrics (input rate, processing rate, batch duration, state size) and the Streaming tab in the Spark UI.
- **Data**: SQL alerts and Lakehouse Monitoring on freshness, volume and quality.
- **Platform**: system tables for audit, billing, lineage, query history and compute usage; Spark UI and cluster metrics for tuning.
- Send notifications to Slack, Teams, email or PagerDuty with clear runbooks, and track SLAs and SLOs on key datasets.

Q30. How does Databricks compare with Snowflake, Synapse or Fabric? How do you position it?

Keep the answer balanced and based on workloads, not on slogans.

Databricks' strengths: open formats (Delta, Iceberg support) on your own storage, a single engine for ETL, streaming, SQL and ML/AI, strong Spark and Python/Scala flexibility, Unity Catalog governance across data and AI, and strong cost-performance for large-scale engineering. Snowflake is traditionally very strong for SQL-first warehousing with simple operations, and both platforms have been moving toward each other (Databricks SQL on one side, Snowpark and Iceberg support on the other). Microsoft Fabric is an integrated SaaS analytics suite tightly tied to the Microsoft and Power BI ecosystem, and it also uses Delta/Parquet storage in OneLake.

Choice depends on team skills, workload mix (heavy engineering and ML versus pure BI), existing cloud and licensing, governance needs and total cost. Product features change quickly, so validate with a proof of concept on your own data.

Q31. What does the Mosaic AI / GenAI side of Databricks include?

At a high level, the platform covers the full AI lifecycle on governed data: **MLflow** for tracking, evaluation and the model registry; **Feature engineering** for reusable features; **Model Serving** for real-time endpoints (custom models, foundation models and external model providers); **Vector Search** for embeddings and retrieval-augmented generation (RAG); **Agent** tooling for building, evaluating and tracing agents; and AI functions in SQL for calling models on tables. Unity Catalog governs models, functions and data used by AI systems. For a data-engineering interview, be able to describe a simple RAG flow: ingest and chunk documents, create embeddings, index them in Vector Search, retrieve, and call an LLM.

Section G: Scenario and Behavioural Questions

Q32. Describe a production incident you handled. How should you structure the answer?

Use the **STAR** format and keep it concrete.

- **Situation**: "A nightly Gold pipeline feeding finance dashboards missed its SLA, and the numbers were wrong."
- **Task**: your responsibility: restore the data quickly and prevent a repeat.
- **Action**: how you triaged (job run history, Spark UI, Delta history), found the root cause (for example an upstream schema change plus a duplicate-key MERGE failure), the immediate fix (restored the table with `RESTORE` and re-ran from Bronze), communicated with stakeholders, and the permanent fix (schema checks, deduplication, alerts, a runbook).
- **Result**: measurable outcomes (recovery time, no repeat in six months, runtime reduced).

Interviewers look for ownership, calm structured debugging, communication and prevention. Prepare two or three real stories in advance, one each on performance, data quality and a design decision.

Q33. Scenario: a source system sends changing JSON with new fields every week. How do you keep the pipeline stable?

- Ingest to Bronze with Auto Loader, with schema evolution (`addNewColumns`) or `rescue` mode, and keep the raw payload and `_rescued_data` so nothing is lost.
- Store semi-structured data in a **VARIANT** column or keep the raw JSON string in Bronze, and parse only the fields you need in Silver with explicit types.
- Treat Silver as the contract: explicit schema, expectations that quarantine bad rows, and alerts on schema change events.
- Agree a data contract with the producer, and add a monitoring check for new or removed fields.
- Auto-restart the stream after a schema-change stop using job retries.

The principle: be flexible at the edge (Bronze), strict at the core (Silver and Gold).

Q34. Scenario: dashboards are slow and the SQL warehouse bill is high. What do you do?

- **Measure**: Query History and the query profile to find the slowest and most frequent queries, scan versus shuffle time, and queue time.
- **Fix the data**: Gold tables pre-aggregated (materialized views), liquid clustering on filter columns, OPTIMIZE and statistics, and removing unnecessary wide joins.
- **Fix the queries and BI**: select only needed columns, push filters, avoid per-user heavy live queries (use cached or scheduled extracts and dashboard caching), and reduce refresh frequency.
- **Fix the warehouse**: serverless for fast start and auto-stop, right-size (bigger size for heavy single queries, more clusters for concurrency), and separate warehouses per workload type so ad-hoc users cannot starve dashboards.
- **Govern cost**: tags, budgets and a review of who runs the most expensive queries.

Q35. Scenario: how do you decide between batch, micro-batch streaming and true real-time?

Start from the business requirement for *freshness* and the cost of being late.

- **Daily or hourly decisions**: scheduled incremental batch (`availableNow` or declarative pipelines triggered), cheapest and simplest.
- **Minutes**: continuous Structured Streaming micro-batches or continuous pipelines.
- **Sub-second** (fraud, personalisation): consider an operational store and a lower-latency path, with Delta as the analytical sink; Databricks streaming is excellent for near-real-time but is not an OLTP system.

Also consider complexity (state, late data, replay), team skills and cost: always-on compute versus periodic compute. Most business needs turn out to be "minutes", not "milliseconds".

Q36. What are common mistakes you have seen on Databricks projects, and how do you avoid them?

- **Running production on all-purpose clusters** (cost and instability): use job compute or serverless.
- **Over-partitioning** small tables: leads to tiny files; use liquid clustering.
- **No maintenance** of Delta tables: enable Predictive Optimization or schedule OPTIMIZE and VACUUM.
- **Using** `collect()` **or** `toPandas()` on big data and Python UDFs where built-ins exist.
- **Ignoring governance early**: retrofitting Unity Catalog and permissions later is painful.
- **Hard-coded paths, secrets and environment details** in notebooks.
- **Non-idempotent pipelines** that duplicate data on retry.
- **No monitoring or cost tags**, so problems are found by the business first.

Each fix is a standard, a template or a guardrail (policies, bundles, code review checklists) rather than relying on memory.

Appendix A: Rapid-Fire Reference (Defaults and Numbers)

Numbers interviewers like to ask. Defaults can change between runtime versions, so treat these as a revision aid and confirm in the docs.

Topic	Value
Spark shuffle partitions	200 (spark.sql.shuffle.partitions)
Auto broadcast join threshold	10 MB (spark.sql.autoBroadcastJoinThreshold)
AQE skew rule (default)	Partition over 5 times the median and over 256 MB
DataFrame cache() level	MEMORY_AND_DISK
VACUUM default retention	7 days (168 hours)
Delta log retention	30 days (delta.logRetentionDuration)
Checkpoint file frequency	Every 10 commits
Statistics collected for	First 32 columns (delta.dataSkippingNumIndexedCols)
OPTIMIZE target file size	Up to about 1 GB per file
When to partition a table	Over about 1 TB, with at least about 1 GB per partition
Liquid clustering keys	Up to 4 clustering columns
Default isolation level	WriteSerializable (stronger option: Serializable)
Auto Loader schema evolution default	addNewColumns
Streaming output modes	append, update, complete
Unity Catalog hierarchy	Metastore, catalog, schema, then table, view, volume, function, model
Metastores	One Unity Catalog metastore per region per account
Change Data Feed change types	insert, update_preimage, update_postimage, delete
Cluster access modes	Standard (shared) and Dedicated (single user or group)
SQL warehouse types	Serverless, Pro, Classic
Pipeline expectation actions	Warn (keep), drop row, fail update

Appendix B: Final Interview Checklist

- Be able to **draw the architecture**: control plane, compute plane, cloud storage, Unity Catalog, Medallion layers.
- Know **Delta internals** well: transaction log, checkpoints, OPTIMIZE, VACUUM, MERGE, liquid clustering.
- Know how to **read the Spark UI** and name the cause of each symptom (skew, spill, small files, shuffle).
- Prepare **three real stories**: one performance win, one data-quality or incident fix, one design decision with trade-offs.
- Be ready to **write code live**: dedup with window functions, MERGE, Auto Loader read, a simple aggregation, a streaming write with a checkpoint.
- For scenario questions, **clarify requirements first** (volume, latency, SLA, governance), then design in layers, then discuss cost and monitoring.
- Be honest about what you have and have not used in production.